TRABAJO INTEGRADOR ABP - MÓDULO PROGRAMADOR

EVIDENCIA 6 - HITO 3

CARRERA: Tecnicatura Superior en Desarrollo de Software

ESPACIO CURRICULAR: Programación y Base de Datos 1

COMISIÓN: B1

Sistema de Gestión de Reparaciones de PC: "TecnoService"

EQUIPO:

- KIEFFER, Ricardo Hernán

- JERONIMO, Juan

- BILEISIS, Joaquín

- ESPINOSA, Thiago

CICLO LECTIVO: 2026

Link Repositorio Git: https://github.com/ricardokieffer/Bit-Fix-TecnoService.git

USUARIOS IDENTIFICADOS:

• Administrador / Encargado del Taller:

- Rol en la Aplicación: Posee acceso y control total sobre el sistema de escritorio. Es responsable de dar de alta o modificar técnicos, administrar el catálogo de repuestos e inventario, supervisar todas las órdenes de trabajo, aprobar presupuestos y consultar los informes de facturación o reparaciones terminadas.

- Mapeo y Permisos SGBD: Se mapea al rol de base de datos 'admin_role' con permisos completos (ALL PRIVILEGES / SELECT, INSERT, UPDATE, DELETE) sobre la totalidad de las tablas del SGBD.

• Técnico Reparador:

- Rol en la Aplicación: Posee un acceso enfocado exclusivamente en las funciones del taller mecánico/informático. Consulta las órdenes de reparación asignadas a su legajo, carga los diagnósticos técnicos, asocia los repuestos del inventario consumidos en la reparación especificando su cantidad y actualiza el estado del servicio (ej. 'En diagnóstico', 'En reparación', 'Terminado').

- Mapeo y Permisos SGBD: Se mapea al rol de base de datos 'tecnico_role' con permisos restringidos: lectura (SELECT) de datos de clientes, equipos y repuestos, e inserción/actualización (INSERT/UPDATE) únicamente en los campos de diagnóstico, estado y detalle de repuestos consumidos en la orden.

• Recepcionista / Operador de Atención:

- Rol en la Aplicación: Se encarga de la atención inicial al cliente en la recepción del taller. Registra los datos del cliente, los datos técnicos del equipo ingresado (tipo, marca, modelo, número de serie), redacta la falla reportada por el usuario, asigna el técnico responsable y genera la orden de trabajo inicial con presupuesto estimado en estado 'Ingresada'.

- Mapeo y Permisos SGBD: Se mapea al rol de base de datos 'recepcion_role' con permisos de lectura e inserción (SELECT, INSERT, UPDATE) sobre las tablas CLIENTE, EQUIPO y REPARACION, previniendo la eliminación de registros o modificaciones en configuraciones críticas.

Operatoria del Taller y Contexto del Sistema

El taller TecnoService opera recibiendo dispositivos informáticos traídos por los clientes. Al ingresar un equipo, se toman los datos de contacto del cliente, los datos identificatorios del hardware y la falla reportada por el usuario. En esta etapa de recepción, se asigna un Técnico Responsable y se establece un Presupuesto Estimado inicial (costo estimado de la revisión/mano de obra preliminar).

Durante la etapa de taller, el técnico diagnostica la falla técnica, actualiza el estado de la reparación (ej: Ingresada → En Diagnóstico → Esperando Repuesto → En Reparación → Terminado → Entregado) y asocia los Repuestos del inventario que fueron necesarios utilizar para concretar el arreglo, indicando la cantidad empleada de cada componente. Finalmente, se calcula el Costo Total (monto a facturar), el cual combina el costo estimado de mano de obra con la suma del precio unitario por cantidad de todos los repuestos consumidos.

ENTIDADES IDENTIFICADAS

• **CLIENTE (Entidad Fuerte):**

- **id_cliente:** Clave Primaria (PK) - Identificador único de cliente.
- **DNI:** Documento Nacional de Identidad.
- **nombre:** Nombre y apellido del cliente.
- **teléfono:** Número telefónico de contacto directo.

• **EQUIPO (Entidad Fuerte):**

- **id_equipo:** Clave Primaria (PK) - Identificador del dispositivo.

- **tipo:** Tipo de hardware (PC de escritorio, Notebook, Impresora).
- **marca:** Marca del fabricante (ej. Lenovo, HP, Asus).
- **modelo:** Modelo específico del equipo.
- **numero_serie:** Número de serie asignado por el fabricante.

• **TECNICO (Entidad Fuerte):**

- **id_tecnico:** Clave Primaria (PK) - Identificador del técnico.
- **DNI:** Documento de identidad del técnico.
- **nombre:** Nombre del profesional.
- **apellido:** Apellido del profesional.
- **especialidad:** especialidad de reparación.
- **estado:** Activo o inactivo.

• **REPARACION (Entidad Asociativa / Fuerte):**

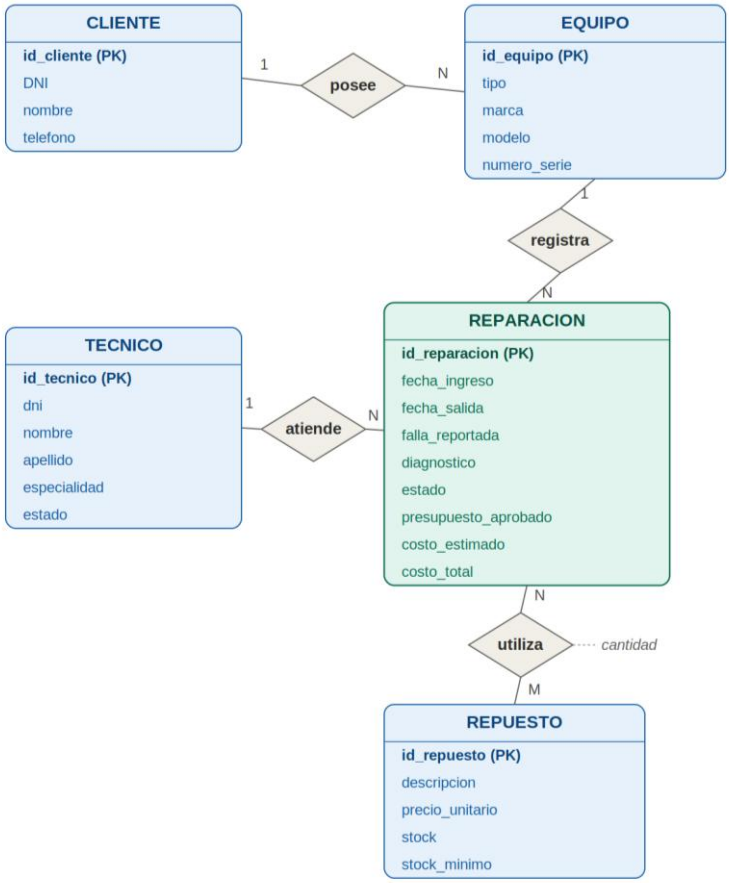
- **id_reparacion:** Clave Primaria (PK) - Número de orden de trabajo.
- **fecha_ingreso:** Fecha de recepción del equipo en el taller.
- **fecha_salida:** Fecha estimada o real de entrega del equipo reparado.
- **falla_reportada:** Descripción del problema informado por el cliente al ingresar el equipo.
- **diagnostico:** Diagnóstico técnico detallado y trabajo realizado por el técnico.
- **estado:** Estado actual de la orden (Ingresada, En Diagnóstico, Esperando Repuesto, En Reparación, Terminado, Entregado).
- **presupuesto_aprobado:** Si el presupuesto resultó aprobado o no.
- **costo_estimado:** Presupuesto inicial estimado asignado al ingresar.
- **costo_total:** Monto final a facturar (mano de obra + repuestos consumidos).

• **REPUESTO (Entidad Fuerte / Catálogo):**

- **id_repuesto:** Clave Primaria (PK) - Código del componente en inventario.
- **descripcion:** Nombre/detalle del repuesto (ej: Disco SSD 480GB).
- **precio_unitario:** Precio de venta unitario del repuesto.

- **stock:** Cantidad de unidades disponibles en stock.
- **stock_minimo:** cantidad mínima antes de avisar que falta stock.

BOCETO DEL MODELO E-R



Presupuesto, Facturación y Repuestos

1. Presupuesto y Facturación: Se determinó técnicamente que costo_estimado (Presupuesto) y costo_total (Facturación final) no requieren ser entidades independientes en esta etapa, ya que mantienen una relación 1:1 estricta con la orden de reparación. Se modelan como atributos de REPARACION, simplificando la estructura sin perder trazabilidad financiera.

2. Relación N:M entre REPARACION y REPUESTO: Dado que una orden de reparación puede requerir múltiples repuestos y un mismo tipo de repuesto puede ser utilizado en muchas reparaciones distintas, la relación utiliza tiene cardinalidad N:M. Esta relación posee el atributo de relación cantidad, que registra exactamente cuántas unidades de un repuesto concreto se utilizaron en esa orden específica.

3. Aprobación del Presupuesto: Se incorporó el atributo presupuesto_aprobado, independiente de costo_estimado, porque responde una pregunta distinta: no cuánto cuesta la reparación, sino si el cliente ya dio el visto bueno para hacerla. Mientras no esté en estado 'Aceptado', el sistema impide que la orden avance a 'En Reparación', 'Terminado' o 'Entregado' (ver Minuta de Requerimientos, punto 3.3).

Reglas de Negocio y Cardinalidades

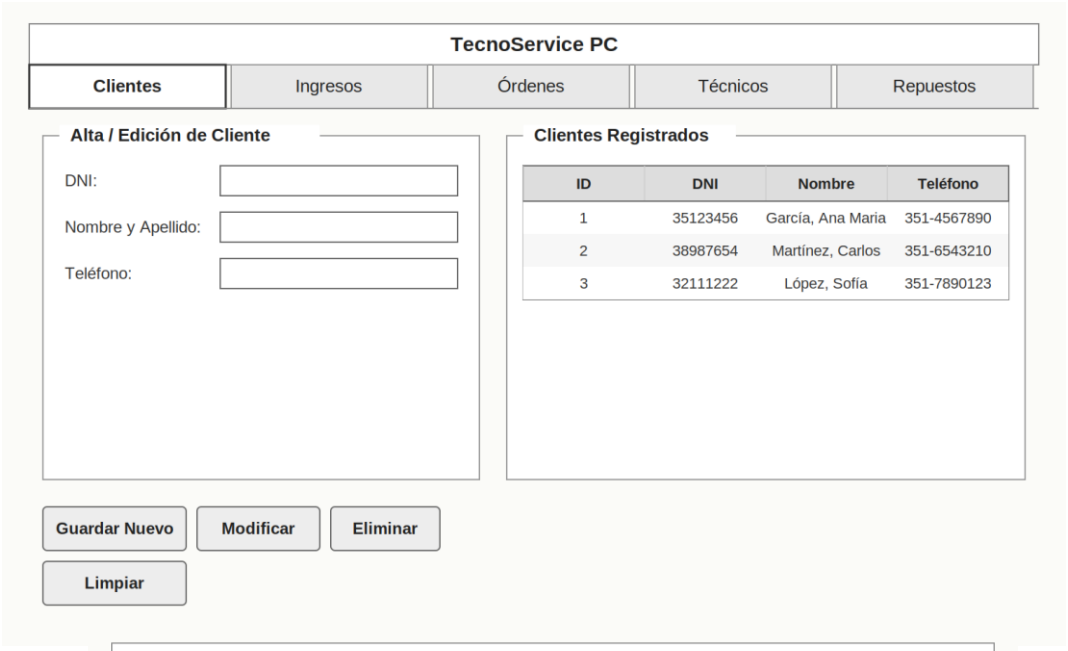
• **CLIENTE (1) —— <posee> —— (N) EQUIPO:** Un cliente puede registrar múltiples equipos en el taller (1:N), pero cada equipo pertenece a un único cliente.

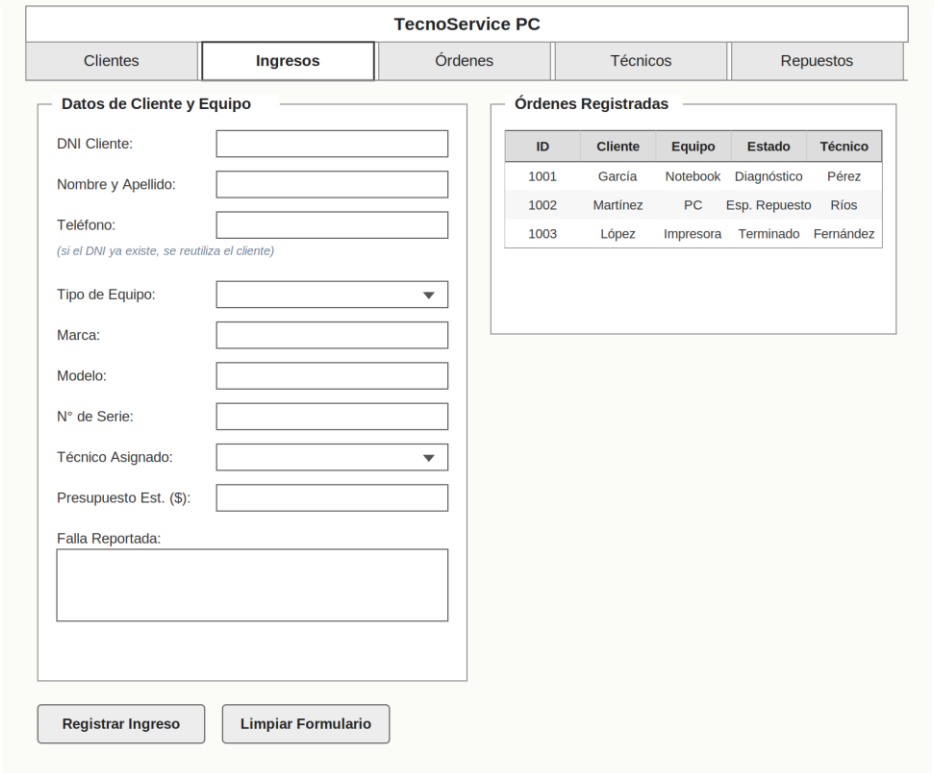
• **EQUIPO (1) —— <registra> —— (N) REPARACION:** Un equipo puede ingresar al taller múltiples veces a lo largo del tiempo por fallas distintas (1:N), pero cada orden de reparación corresponde a un único equipo.

• **TECNICO (1) —— <atiende> —— (N) REPARACION:** Un técnico puede atender múltiples órdenes de reparación en el taller (1:N), pero cada orden tiene asignado un único técnico responsable.

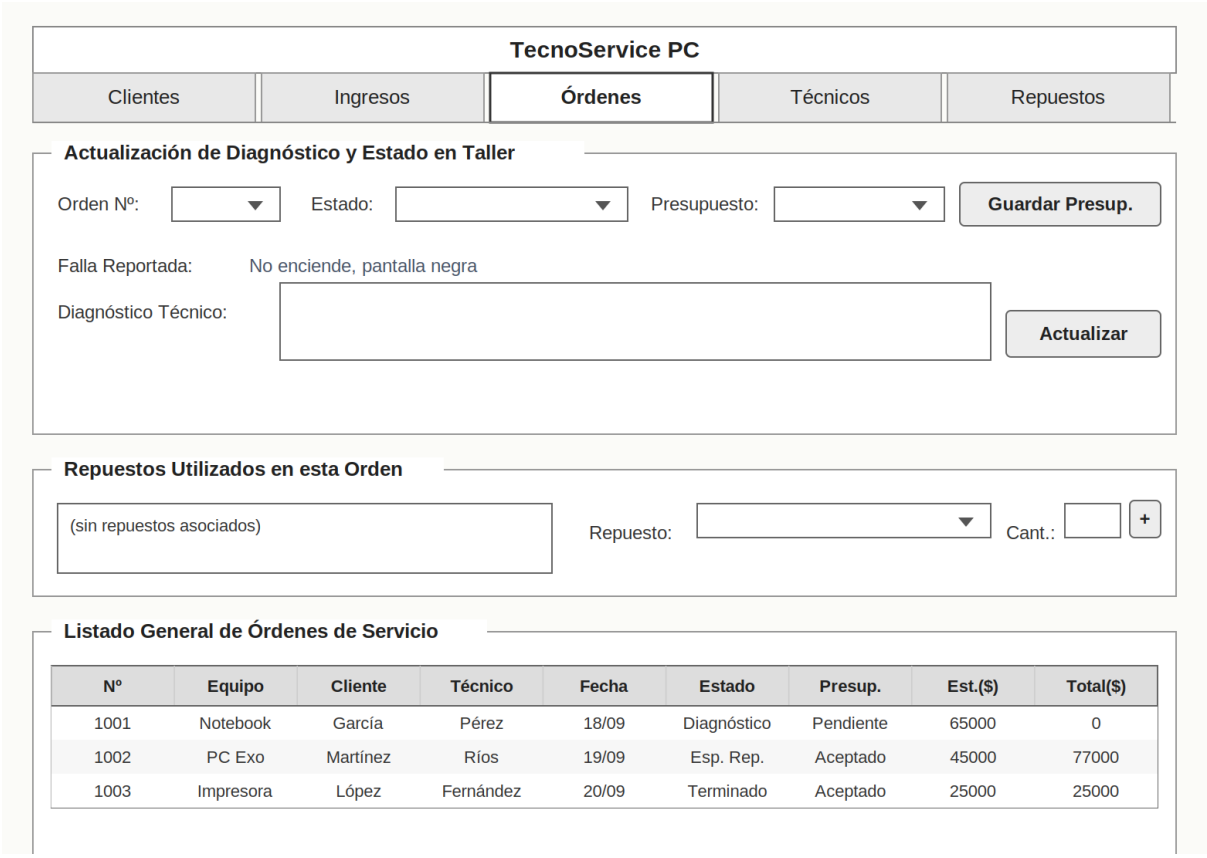
• **REPARACION (N) —— <utiliza> —— (M) REPUESTO:** Una reparación utiliza N repuestos y un repuesto se utiliza en M reparaciones. La relación posee el atributo 'cantidad'.

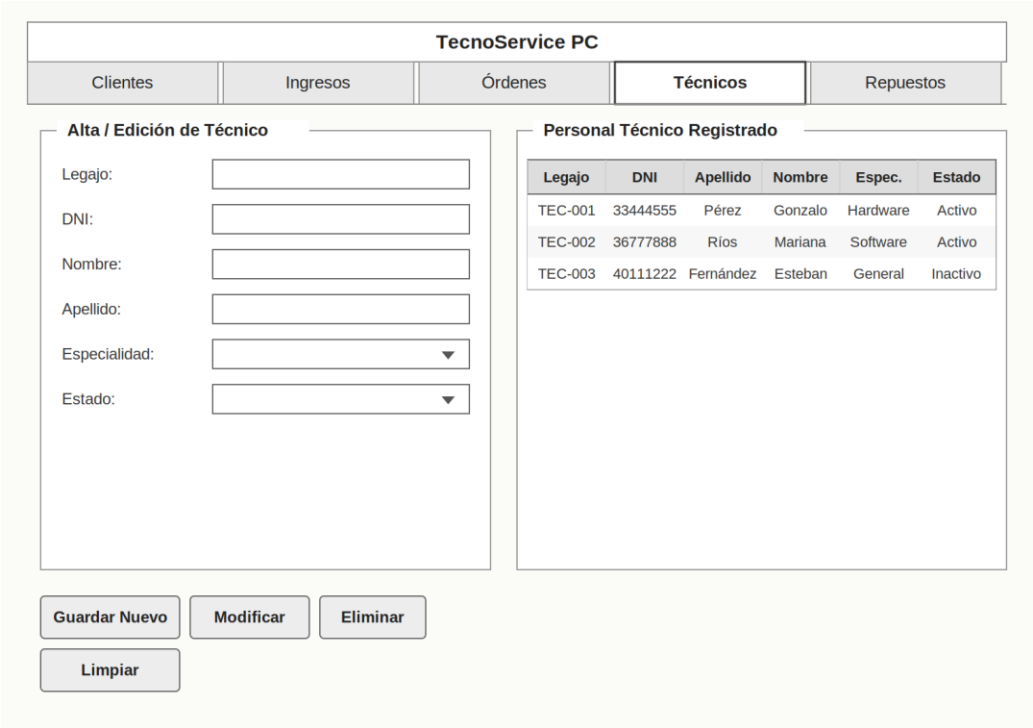
BOCETOS DE LAS PANTALLAS



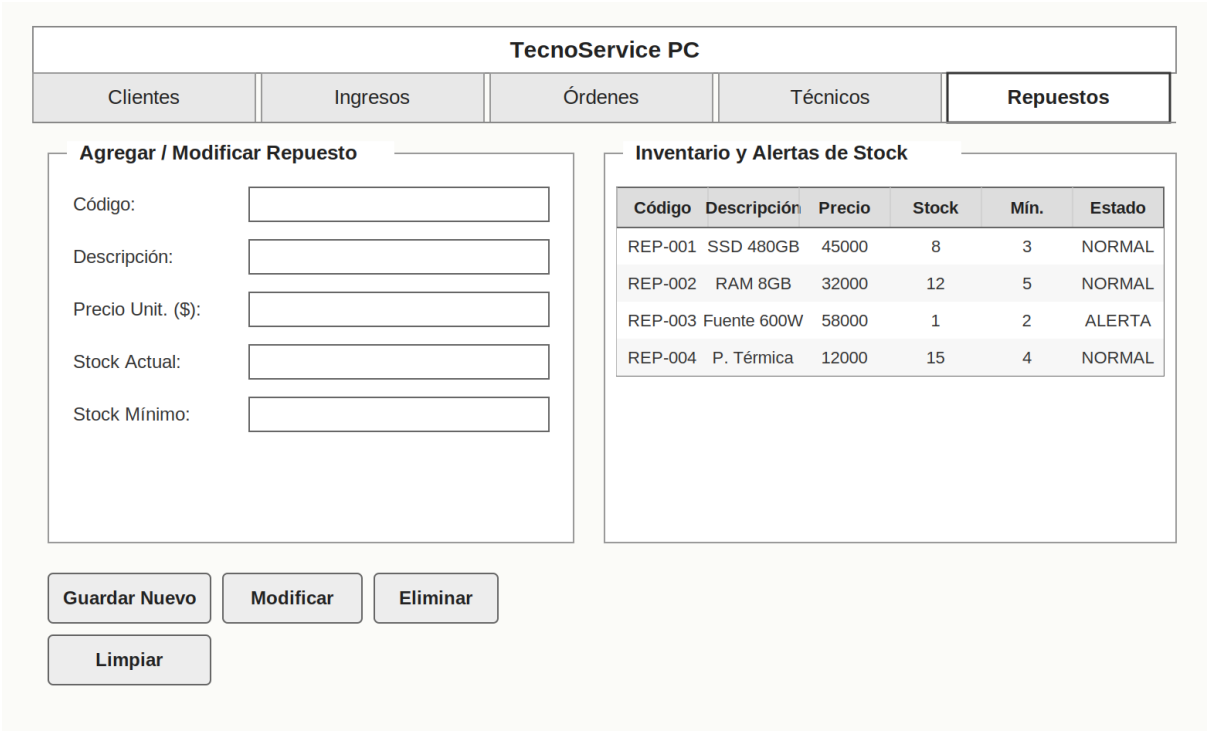


ISPC
INSTITUTO SUPERIOR POLITÉCNICO CÓRDOBA





Dirección General de EDUCACIÓN TÉCNICA Y FORMACIÓN PROFESIONAL
Ministerio de EDUCACIÓN
CÓRDOBA Seguimos haciendo



CONFORMACIÓN DEL EQUIPO Y ASIGNACIÓN DE ROLES

Integrante	Rol Principal	Responsabilidad en el Proyecto
KIEFFER, Ricardo	Coordinador & Validaciones	Coordinación general del equipo, control de plazos, validación de formularios de entrada en Tkinter y gestión de excepciones en base de datos.
JERONIMO, Juan	Modelo de Datos & Acceso SQL	Diseño del Modelo E-R, normalización hasta 3FN, diccionario de datos, script DDL y desarrollo de consultas SQL parametrizadas.
BILEISIS, Joaquin	Interfaz Gráfica (UI)	Construcción de la interfaz de escritorio en Tkinter utilizando ttk.Notebook (5 pestañas), ttk.LabelFrame y listados tabulares con ttk.Treeview.

ESPINOSA, Thiago	Documentación y Pruebas	Registro de la bitácora semanal, redacción del informe técnico final, elaboración del manual de usuario y ejecución de pruebas de integración.

Presentación y Repositorio Git

El presente informe documenta la entrega del Hito 3 (Evidencia 6) del proyecto TecnoService. Incluye el modelo de datos normalizado en 3FN, el script con roles y datos de prueba, el diccionario de datos y la aplicación en Python con Tkinter conectada a PostgreSQL.

El repositorio del grupo tiene commits de todos los integrantes.

Motor de Base de Datos

El proyecto utiliza PostgreSQL como motor relacional, conforme a los Requisitos Técnicos Obligatorios de la cátedra ("Motor relacional: PostgreSQL o MySQL"). El script script_tecnoservice.sql está escrito íntegramente en sintaxis de PostgreSQL (tipos SERIAL, bloques DO $$...$$ para la creación condicional de roles), y el módulo conexion.py se conecta con la librería psycopg2 a la base en PostgreSQL.

Estructura Modular del Código Python (Tkinter)

Siguiendo los requisitos de la cátedra, el código se encuentra dividido modularmente en 4 archivos principales:

- main.py: Punto de entrada principal que ejecuta la aplicación y el bucle de eventos tk.Tk().
- interfaz.py: Construye la ventana de escritorio con ttk.Notebook y las 5 pestañas navegables con formularios, botones y listas tabulares ttk.Treeview.

- acceso_datos.py: Ejecuta las sentencias SQL (altas, consultas, modificaciones y bajas) contra la base.
- conexion.py: Abre la conexión a PostgreSQL con psycopg2.

Coherencia del Modelo de Datos

El modelo de datos separa claramente los datos del hardware (tabla equipo) de los datos de cada visita al taller (tabla reparación). La falla reportada por el cliente y el estado del proceso se registran en reparación y no en equipo, porque un mismo equipo puede ingresar más de una vez al taller con fallas distintas en cada oportunidad (cardinalidad equipo 1:N reparación). Este criterio es el que sigue el diccionario de datos, el script DDL y el prototipo Tkinter, de forma consistente entre los tres.

Los dominios de valores fijos (tipo de equipo, especialidad del técnico y estado de la reparación) están unificados entre el script SQL (restricciones CHECK), el diccionario de datos y los combobox de la interfaz, para que un dato cargado desde la aplicación no viole una restricción de la base.

Roles de Acceso a Nivel de Base de Datos

Conforme al apartado "Alcance: Qué No Se Solicita" de los lineamientos, el proyecto no implementa un sistema de login ni gestión de sesiones a nivel de aplicación. El control de acceso se resuelve exclusivamente a nivel del SGBD mediante tres roles de PostgreSQL creados en el script DDL:

- admin_role: control total sobre todas las tablas y secuencias.
- tecnico_role: lectura general y actualización de diagnóstico, estado y repuestos utilizados en las reparaciones.

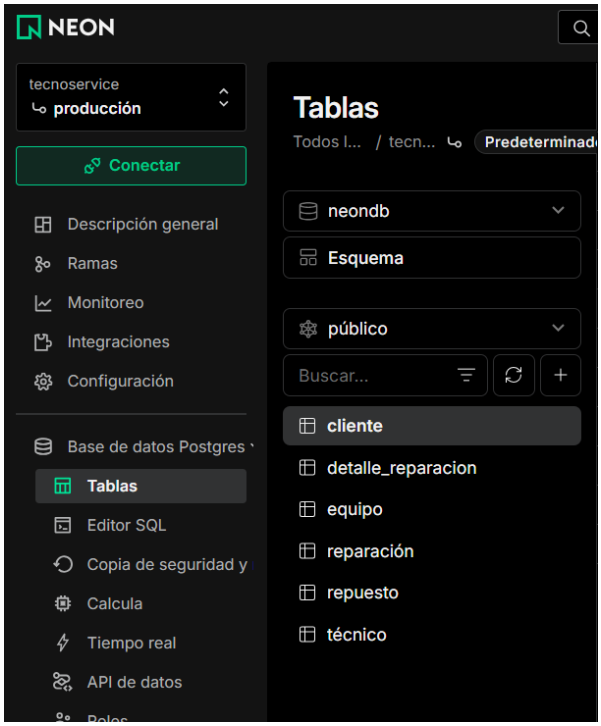
- recepcion_role: alta de clientes y equipos, y creación de órdenes de trabajo.

Estos tres roles son coherentes con los tres tipos de usuario identificados en el relevamiento del sistema (Administrador, Técnico y Recepcionista).

IMPLEMENTACIÓN EN POSTGRESQL (HITO 3)

Base de datos implementada:

La base de datos se creó en PostgreSQL, usando el servicio en la nube Neon. Se ejecutó el script script_tecnoservice.sql, que crea las 6 tablas (cliente, equipo, tecnico, reparacion, repuesto y detalle_reparacion) y carga 10 registros de prueba en cada tabla principal.



Roles e integridad

El script crea tres roles: admin_role, tecnico_role y recepcion_role. A cada uno se le dieron permisos con GRANT:

- admin_role tiene todos los permisos sobre las tablas.
- tecnico_role puede leer clientes, equipos y repuestos, y puede insertar y modificar reparaciones y el detalle de repuestos.
- recepcion_role puede leer, insertar y modificar clientes, equipos y reparaciones, y solo leer técnicos y repuestos.

Para la integridad de los datos se usaron estas restricciones:

- Claves primarias en todas las tablas. En detalle_reparacion la clave es compuesta (id_reparacion e id_repuesto).
- Claves foráneas entre equipo y cliente, reparacion y equipo, reparacion y tecnico, y detalle_reparacion con reparacion y repuesto.
- CHECK para los valores fijos: tipo de equipo, especialidad y estado del técnico, estado de la reparación y presupuesto_aprobado. También para que precios, stock y cantidades no sean negativos.
- Un CHECK que impide que una orden pase a “En Reparación”, “Terminado” o “Entregado” si el presupuesto no está en “Aceptado”.
- Un trigger (trg_validar_stock) que se ejecuta antes de cada INSERT en detalle_reparacion. Controla que el repuesto tenga stock suficiente y, si alcanza, descuenta la cantidad usada. Si no alcanza, rechaza la operación con un mensaje de error.

Conexión desde Python

El archivo conexion.py usa la librería psycopg2 para conectarse a PostgreSQL. Tiene la clase ConexionBD con los datos de la base (host, usuario, contraseña y nombre de la base) y el método obtener_conexion(), que devuelve la conexión. La conexión usa SSL, que es lo que pide Neon.

El archivo acceso_datos.py tiene la clase AccesoDatosSQL. Cada método abre una conexión, ejecuta una sentencia SQL propia y la cierra. Los valores se pasan como parámetros con %s en vez de armar el texto de la consulta a mano, para evitar errores y problemas de seguridad. Para guardar los cambios se usa commit, y si hay un error se hace rollback.

Alta y consulta desde la interfaz

Desde la pestaña Ingresos se carga un cliente, su equipo y la orden de reparación. Al apretar “Registrar Ingreso” se guardan los tres registros en la base, y la orden aparece enseguida en el listado y en la pestaña Órdenes. Esos datos también se pueden ver en Neon con un SELECT. Las pestañas Clientes, Técnicos y Repuestos tienen alta, modificación, eliminación y listado.

Integridad en acción

Si en la pestaña Órdenes se intenta pasar una orden a “En Reparación”, “Terminado” o “Entregado” con el presupuesto en “Pendiente” o “Rechazado”, la base rechaza el cambio y la aplicación muestra un mensaje de error. Para que se pueda avanzar, primero hay que poner el presupuesto en “Aceptado”. Algo parecido pasa al eliminar un técnico o un repuesto que ya se usó en una orden: la clave foránea no lo permite y se muestra un aviso.

El control de stock se puede probar en la base. Si se inserta en detalle_reparacion una cantidad mayor al stock del repuesto, el trigger rechaza la operación con el mensaje “Stock insuficiente”. En esta etapa el stock se controla desde la base. La pantalla para cargar repuestos en una orden queda para la entrega final.

Cómo probarlo

1. Instalar la librería con pip install psycopg2-binary.
2. Abrir una terminal en la carpeta src del proyecto y ejecutar python main.py.
3. La base está en Neon y ya tiene los datos cargados. Los datos de conexión están en conexion.py, así que no hace falta instalar PostgreSQL.

ISPC
INSTITUTO SUPERIOR
POLITÉCNICO CÓRDOBA

INSTITUTO SUPERIOR POLITÉCNICO CÓRDOBA

DICCIONARIO DE DATOS

TABLA: cliente

Almacena la información de contacto e identificación de los clientes del taller.

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_cliente	INT	PK	NOT NULL, SERIAL	Identificador único de cliente (Clave Primaria).
dni	VARCHAR(15)	-	NOT NULL, UNIQUE	Documento Nacional de Identidad del cliente.
nombre	VARCHAR(100)	-	NOT NULL	Nombre y apellido completo del cliente.
telefono	VARCHAR(30)	-	NOT NULL	Número telefónico de contacto principal.

TABLA: equipo

Registra los dispositivos e impresoras que pertenecen a un cliente. Solo contiene datos del hardware en sí: la falla reportada y el estado de cada visita al taller se registran en la tabla reparacion, porque un mismo equipo puede ingresar varias veces por fallas distintas (cardinalidad 1:N con reparacion).

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_equipo	INT	PK	NOT NULL, SERIAL	Identificador único del equipo (Clave Primaria).
tipo	VARCHAR(30)	-	NOT NULL, CHECK (tipo IN ('Notebook','PC de Escritorio','Impresora','All in One','Otro'))	Tipo de hardware.
marca	VARCHAR(50)	-	NOT NULL	Marca del fabricante (ej. Lenovo, HP, Asus).
modelo	VARCHAR(50)	-	NOT NULL	Modelo específico del equipo.
numero_serie	VARCHAR(50)	-	UNIQUE	Número de serie asignado por el fabricante.

id_cliente	INT	FK	NOT NULL, REFERENCES cliente(id_cliente)	Clave foránea que vincula con el cliente dueño.

TABLA: tecnico

Mantiene el registro de los técnicos profesionales habilitados en el taller.

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_tecnico	INT	PK	NOT NULL, SERIAL	Identificador único del técnico (Clave Primaria).
legajo	VARCHAR(20)	-	NOT NULL, UNIQUE	Código de legajo interno del taller (ej: TEC-001), distinto del DNI.
dni	VARCHAR(15)	-	NOT NULL, UNIQUE	Documento de identidad del profesional.
nombre	VARCHAR(50)	-	NOT NULL	Nombre del técnico.
apellido	VARCHAR(50)	-	NOT NULL	Apellido del técnico.
especialidad	VARCHAR(50)	-	NOT NULL, CHECK (especialidad IN ('Hardware','Software','Electrónica','General'))	Especialidad principal del técnico.
estado	VARCHAR(20)	-	DEFAULT 'Activo', CHECK (estado IN ('Activo','Inactivo'))	Disponibilidad operativa.

TABLA: reparacion

Entidad central del sistema: la orden de trabajo. Guarda la falla reportada por el cliente al momento del ingreso, el diagnóstico técnico, el estado del proceso, las fechas y los costos.

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_reparacion	INT	PK	NOT NULL, SERIAL	Número correlativo de orden de trabajo (Clave Primaria).
fecha_ingreso	DATE	-	NOT NULL, DEFAULT CURRENT_DATE	Fecha de recepción del equipo en el taller.
fecha_salida	DATE	-	CHECK (fecha_salida IS NULL OR fecha_salida >= fecha_ingreso)	Fecha real o estimada de entrega.

falla_reportada	TEXT	-	NOT NULL	Descripción del problema informado por el cliente al ingresar el equipo.
diagnostico	TEXT	-	NULL	Diagnóstico técnico detallado y trabajo realizado.
costo_estimado	DECIMAL(10,2)	-	DEFAULT 0.00, CHECK (costo_estimado >= 0)	Presupuesto inicial estimado ($).
costo_total	DECIMAL(10,2)	-	DEFAULT 0.00, CHECK (costo_total >= 0)	Liquidación final acumulada ($).
estado	VARCHAR(30)	-	DEFAULT 'Ingresada', CHECK (estado IN ('Ingresada','En Diagnóstico','Esperando Repuesto','En Reparación','Terminado','Entregado'))	Estado actual de la orden.
presupuesto_aprobado	VARCHAR(20)	-	DEFAULT 'Pendiente', CHECK (presupuesto_aprobado IN ('Pendiente','Aceptado','Rechazado')). Además, CHECK a nivel de tabla: no puede haber estado IN ('En Reparación','Terminado','Entregado') si este campo no es 'Aceptado'.	Indica si el cliente aceptó, rechazó, o todavía no respondió el presupuesto (ver Minuta de Requerimientos, punto 3.3).
id_equipo	INT	FK	NOT NULL, REFERENCES equipo(id_equipo)	Equipo asociado a la orden.
id_tecnico	INT	FK	NOT NULL, REFERENCES tecnico(id_tecnico)	Técnico asignado responsable.

TABLA: repuesto

Almacena el inventario de componentes y piezas de reemplazo (catálogo de insumos).

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_repuesto	INT	PK	NOT NULL, SERIAL	Identificador interno del componente (Clave Primaria).

codigo	VARCHAR(20)	-	NOT NULL, UNIQUE	Código de catálogo del repuesto (ej: REP-001), usado de cara al usuario.
descripcion	VARCHAR(150)	-	NOT NULL	Detalle/nombre del repuesto (ej: Disco SSD 480GB).
precio_unitario	DECIMAL(10,2)	-	NOT NULL, CHECK (precio_unitario > 0)	Precio de venta unitario ($).
stock	INT	-	NOT NULL, DEFAULT 0, CHECK (stock >= 0)	Cantidad física disponible en almacén.
stock_minimo	INT	-	NOT NULL, DEFAULT 2, CHECK (stock_minimo >= 0)	Umbral para alerta de reabastecimiento.

TABLA: detalle_reparacion

Tabla intermedia que resuelve la relación N:M entre reparacion y repuesto, registrando qué repuestos (y en qué cantidad) se usaron en cada orden. La clave primaria es compuesta: no existe una columna id_detalle propia.

Columna / Campo	Tipo de Dato	Clave	Restricciones	Descripción / Regla
id_reparacion	INT	PK, FK	NOT NULL, REFERENCES reparacion(id_reparacion) ON DELETE CASCADE	Orden de reparación asociada. Forma parte de la clave primaria compuesta.
id_repuesto	INT	PK, FK	NOT NULL, REFERENCES repuesto(id_repuesto) ON DELETE RESTRICT	Repuesto utilizado. Forma parte de la clave primaria compuesta.
cantidad	INT	-	NOT NULL, DEFAULT 1, CHECK (cantidad > 0)	Unidades consumidas en la orden.
precio_unitario_aplicado	DECIMAL(10,2)	-	NOT NULL, CHECK (precio_unitario_aplicado > 0)	Precio vigente al momento de usar el repuesto (histórico, independiente del precio actual del catálogo).

Trigger asociado: trg_validar_stock (función fn_validar_y_descontar_stock) se ejecuta antes de cada INSERT en detalle_reparacion: verifica que repuesto.stock sea

suficiente para la cantidad solicitada (si no, rechaza la operación con un error) y, si es válida, descuenta automáticamente esa cantidad del stock del repuesto. Resuelve la regla de negocio de la Minuta de Requerimientos, punto 3.2.